Evaluating GPT-4o-mini for Unit Test Generation on Java and Python Functions of Medium Cyclomatic Complexity

Nguyễn Tùng Ân, Đoàn Thị Thu Kim, Sơn Hải, Trương Hoàng Phúc, Trần Xuân Lộc
Group 2 — SE1944, FPT University
Supervisor: L.T.Q.Chi

Abstract

Unit-test generation with large language models (LLMs) is becoming increasingly common, but most existing studies still rely on simple, curated benchmarks that neither reflect the complexity of real-world code nor control for it. This study tests the one-shot capability of `gpt-4o-mini` on 120 real-world functions (60 Java, 60 Python) of medium cyclomatic complexity (5–10), mined from ten open-source repositories at pinned commits and evaluated directly inside their original projects. Its results are compared against EvoSuite and Randoop (Java) and Pynguin (Python) on branch coverage and mutation score, using non-parametric tests ($\alpha = 0.05$). Across all 120 functions, median coverage and median mutation score are both 0%. Only 28/120 suites (23.3%) compile and actually test the intended function; their median coverage is 75.0%, still below the 80% bar. Over suites with a measurable mutation score, the median is 36.84% for Python and 0% for Java, both under the 60% threshold. Against the baselines, `gpt-4o-mini` is beaten outright by both Java tools (EvoSuite and Randoop, $r = -1.00$ for each). It beats Pynguin only because Pynguin failed completely on this dataset (median coverage 0.0%). Within the CC 5–10 band there is no clear link between complexity and test quality; what decides the outcome is whether a suite compiles and tests the right function. Python failures are mostly bad imports, while Java calls the wrong method without any error — something only visible when running on real project code. Overall, one-shot LLM generation, without retrieval or repair, cannot yet replace automated test-generation tools on real code of medium complexity. The results support adding feedback loops and richer context when generating tests.

I. Introduction

At first glance, automated unit test generation using large language models seems nearly solved. For example, GPT-4o achieves 98.65% line coverage on TestEval [1], and GPT-4-class model panels report similar numbers on HumanEval-derived suites [2]. The functions behind those numbers, however, are short, self-contained, LeetCode-style problems. A test that performs well under these conditions has never had to deal with a real import graph, navigate a class hierarchy, or handle overloaded method signatures, and high line coverage alone does not guarantee that the test will actually catch faults. When we instead sampled 120 functions from ten real projects — from Apache Commons, Gson and Joda-Time to Flask and Requests — we found that most of the test suites generated by `gpt-4o-mini` (92 out of 120, or 76.7%) never actually exercised the function they were supposed to test.

There are two gaps in current measurement practices that hide this problem. First, hardly any study controls for the structural complexity of the code under test: results are reported in aggregate, across functions with mixed and often unreported cyclomatic complexity (CC), so no one knows at what point LLM-generated tests start to fail (GAP-T). Because of this gap, we kept cyclomatic complexity fixed as a controlled variable, sampled only the 5–10 band, and measured the CC–quality correlation directly (RQ3). Second, branch coverage and mutation score are rarely combined in a single statistical test on a controlled sample (GAP-M), even though coverage alone can overstate quality — in our own baseline data, one `jsoup` function reaches 60% branch coverage while killing 0% of mutants (Section V).

Our design choices were shaped as much by lessons learned during implementation as by insights from the literature. The original proposal named Defects4J and CodeXGLUE as data sources; during preparation we found that CodeXGLUE is a code intelligence benchmark, not a corpus of executable functions, so we mined the dataset ourselves: 120 functions (60 Java, 60 Python) at pinned commits, all with a cyclomatic complexity between 5 and 10, as measured by Lizard. A pilot run then showed that functions extracted from their modules often cannot even resolve the names they reference — in fact, 10 out of our 12 Python pilot measurements were invalid for extraction-related reasons — so in the final experiment every generated test runs inside its original project. This measurement approach is important: measured this way, a score can no longer be reduced by artefacts from extraction, only by the quality of the generated tests themselves. We compare `gpt-4o-mini` (using one-shot prompting, temperature 0) against three automated baselines: EvoSuite and Randoop for Java (60 seconds per class), and Pynguin for Python (90 seconds per module), scoring branch coverage and mutation score, using non-parametric statistical tests with $\alpha = 0.05$.

We ask:

RQ1: Does GPT-4o-mini achieve at least 80% branch coverage on medium-complexity functions?

RQ2: Does GPT-4o-mini achieve at least 60% mutation score, and does it outperform automated baselines?
RQ3: Is there a negative correlation between cyclomatic complexity and test quality?

These two thresholds are not arbitrary: 80% is the branch coverage target most commonly quoted as the industry standard, while 60% is set intentionally above the roughly 34% mutation scores reported for LLM-generated suites in the studies reviewed in Section II; both were fixed in the approved proposal before any data were collected.

In summary, our contributions are: (i) a reproducible benchmark of 120 medium-complexity functions with full provenance; (ii) a measurement pipeline that scores tests *per function* inside the original projects and enforces a green-on-original check before any mutation analysis; (iii) an empirical comparison of GPT-4o-mini against search-based and random baselines on a complexity-matched sample; and (iv) a catalogue of the measurement pitfalls we encountered along the way — such as EvoSuite returning exit code 0 without generating a single test, or JaCoCo silently reporting 0% under EvoSuite's instrumenting class loader — which would have corrupted our results if left unnoticed.

Section II reviews prior work; Section III details the dataset, generation, and measurement process; Section IV answers each research question; Section V interprets our findings, including two exploratory follow-ups; Section VI discusses threats to validity; Section VII summarises our work.

II. Related Work

Three gaps motivate our design. GAP-T is the central gap: plain one-shot GPT-4-family test generation is rarely evaluated at the function level, no prior study jointly examines Java and Python, and existing work does not identify where performance begins to degrade along the complexity axis. GAP-M is narrower but consequential: coverage and fault detection are seldom analysed together in a single controlled experiment, leaving unclear whether high-coverage suites are actually effective. GAP-D follows from both concerns: without explicit complexity constraints, datasets can overrepresent easy, synthetic functions and inflate reported performance. The rest of this section is organised around these three gaps.

A. LLM-based test generation on curated benchmarks

The strongest results in this area come from curated, self-contained benchmarks. Wang et al. [1] introduce TestEval — Python LeetCode-style tasks — and report up to 98.65% line and 97.16% branch coverage for GPT-4o. Kumar et al. [2] reach 99.05% branch coverage on HumanEval-Java across Gemini, GPT-4o, and Claude-3.5. Lira et al. [3] find a 90.69% success rate for Gemini 1.5-Pro when both source and docstring are provided. Chudic and Çalıklı [4] and Jasti et al. [5] push further on similar tasks.

These numbers are hard to argue with — on these benchmarks. The problem is that benchmark functions are short, self-contained, and solved in isolation: the model never encounters real imports, cross-module dependencies, or ambiguous API surfaces. None of these studies reports cyclomatic complexity, so there is no way to tell at what point performance starts to degrade. The benchmarks are also widely used enough that many tasks may have appeared in training data. Our reaction was not "LLMs are great at testing" but "LLMs are great at solving programming puzzles" — a meaningfully different claim.

To study that directly, we built a dataset of 120 functions from ten pinned open-source repositories, restricted to a medium complexity band ($5 \leq \mathrm{CC} \leq 10$, measured with Lizard). The band was fixed in the proposal, before any test was generated, and its point is not that 5–10 is a magic cutoff. It is a deliberate middle ground: complex enough that tests have to exercise real branches and behaviours, and low enough to stay clear of the extreme territory for which our survey found no complexity-controlled, GPT-4-family numbers at all.

B. Degradation on real-world code

Move from curated puzzles to real projects, and the picture changes fast. Haratian et al. [6] find GPT-4o compiling in only 37% of Java pull requests — fail-to-pass at 13%, against EvoSuite's 36% on the same PRs. Sapozhnikov et al. [7] report more than half of ChatGPT-generated Java tests are malformed or non-compiling, partly because prompt-size limits force dropped context. Zhang et al. [8] put compile-error rates at 47.9–55.9% across nine projects for CodeLlama, GPT-3.5, and GPT-4. Konstantinou et al. [9] show the same pattern for a plain zero-shot pipeline — `gpt-4o-mini` among the models — across six Java projects, with an aggregate mutation score down to 33.82%. On Python, Jain and Le Goues [10] report 84.3% pass@1 in the TestForge setting — an agentic pipeline built on GPT-4o — but mutation scores are much lower, which is the more important warning for test quality.

Across these studies, the direction is consistent: real-project code exposes failure modes that curated benchmarks do not. The exact numbers are difficult to compare, however, because each study uses different repositories, prompts, and validity criteria. Haratian et al. [6] provide the closest reference point for our setting because they hold the codebase constant and compare GPT-4o directly against EvoSuite, making the observed gap more attributable to the tool than to dataset choice.

Our own numbers are at the pessimistic end. Java: 91.7% INVALID-or-no-touch. Python: 46.7% INVALID. We count a suite as *effective* only if it compiles, runs, and executes at least one line of the intended target function. A suite that compiles against the wrong method counts as zero. We saw this often enough that it stopped feeling like an edge case. The first time it happened, the suite looked completely plausible — clean compilation, reasonable test names. Only when we checked coverage and mutation did we realise it had never touched the target method at all.

C. Automated (non-LLM) baselines

The standard non-LLM baselines are EvoSuite [11] (genetic search, the default SBST reference on Defects4J [12], [13], [14]), Randoop [15] (feedback-directed random testing), and Pynguin [16] (DYNAMOSA, the Python equivalent). None

relies on natural-language prompting, which is precisely why compile-time failures do not appear in their scores — an asymmetry that matters when interpreting the RQ2-B comparisons.

A fair comparison required standardising the search budgets. In early runs, EvoSuite appeared substantially stronger than Randoop, but the difference was partly caused by configuration: EvoSuite had 60 seconds per class, whereas Randoop had only 10. We therefore standardised the budget to 60 seconds per class for both Java tools and 90 seconds per module for Pynguin. This adjustment ensures that the RQ2-B comparison reflects tool behaviour rather than unequal runtime settings; in the full run, with equal budgets, EvoSuite still led — 55.0 versus 21.1 median branch coverage.

D. Closing the one-shot gap: repair loops, retrieval, and context

When plain prompting fails, the standard response is to add scaffolding. Repair loops feed compiler errors or test failures back to the model — Logic-CoT [17], template-based repair [13], and YATE [18] all follow this pattern, while DISTINCT [19] steers regeneration through description-guided consistency analysis rather than raw compiler output. Retrieval-augmented approaches substitute raw source with real API signatures or class skeletons extracted from call-graph context [20]. Other systems optimise or select over generated candidates — TestDecision, for instance, drives sequential suite construction with greedy optimisation and reinforcement learning [21] — but once that machinery is added, the result is no longer a clean measurement of one-shot generation.

These systems produce measurable improvements. But the improvements are hard to interpret: each layer of scaffolding makes it less clear what the underlying model contributes versus what the surrounding machinery fixes. The cleaner question — how much does one-shot generation achieve on its own, on medium-complexity real code? — has not been answered. That is the missing baseline, and it is what RQ1–RQ3 measure directly.

E. Coverage vs. mutation score as a validity concern

`JA-008` made this problem concrete: a *baseline* suite with 60% branch coverage and a 0% mutation score. We checked the pipeline twice. Nothing was broken. The suite was executing the function — it just never verified the output. Coverage measures execution; mutation score measures whether the tests would catch a fault. The two are not the same thing, and on `JA-008` they pulled in opposite directions.

Moradi Dakhel et al. [22] and Antal et al. [23] both document this pattern: high coverage, near-zero mutation score, tests that visit lines without catching bugs. Jain and Le Goues [10] report the related 84.3% pass@1 / 33.8% mutation-score gap in the TestForge setting. `JA-008` is ours, and it is why we report coverage and mutation score together rather than treating one as a substitute for the other.

Most prior work reports one metric or the other — rarely both, almost never tied together statistically on the same functions — and even when mutation is reported, it is sometimes measured with custom, non-standard harnesses [24] (GAP-M). That gap makes it difficult to say whether LLM-generated tests are genuinely useful or merely good at executing code. We address it directly: both metrics, same function IDs, threshold tests and per-function baseline comparisons run together.

F. Positioning

Reading across everything we surveyed, three things stand out. On curated benchmarks, results are strong — near-perfect coverage on TestEval and HumanEval [1], [2], [3]. On real projects, they are not — Java compilation is often below about 50% [6], [8], and mutation scores sit far below coverage. And most of the real gains in recent work come from the surrounding system — repair loops, retrieval, and selection — not from better raw one-shot generation [17], [19], [13], [18], [20].

What that leaves open is the question we actually care about: how does plain one-shot generation perform on medium-complexity, real-project code, measured consistently across languages and metrics? Nobody has answered that yet. RQ1–RQ3 are our attempt. The setup is deliberately simple — one prompt, one sample, fixed complexity range, same metrics for Java and Python. Simple enough to reason about. Close enough to practice that the failures should matter.

III. Methodology

A. Dataset

The proposal identified Defects4J and CodeXGLUE as data sources. However, CodeXGLUE was ultimately excluded during preparation because it is a code-intelligence benchmark for tasks like summarisation and translation, rather than a corpus of executable functions suitable for coverage or mutation tools. Additionally, no paper in our review had paired it with such measurements. We documented the change as amendment v1.2 (finalised by the team on 2026-07-02, before the full experiment), and collected the functions ourselves from ten repositories at fixed commits. Eight are Java projects overlapping Defects4J's subject programs — `commons-cli`, `commons-math`, `commons-csv`, `commons-collections`, `gson`, `jsoup`, `joda-time`, `jfreechart` — plus `flask` and `requests` for Python. The URL, licence, commit hash, and download date are recorded for every repository, enabling the exact dataset to be reconstructed. The final sample contains **120 real-world functions**, 60 per language, each with a cyclomatic complexity between 5 and 10, as measured by Lizard. One caveat is that `flask` contributes 48 of the 60 Python functions, a by-product of how the two projects are structured. We retained this imbalance — resampling after measurement had begun would have been a greater issue — and disclose it in Section VI.

B. Test generation

LLM. All 120 suites come from one model call each: `gpt-4o-mini-2024-07-18` (the dated snapshot, so reruns hit identical weights), `temperature=0`, `top_p=1`, `max_tokens=2048`, one-shot — a single worked example

precedes the target function in every prompt. The proposal had named `gpt-4o`; the team's API account simply had no access to it when the experiment ran, so we switched to `gpt-4o-mini` and recorded the change as amendment v1.1 (documented 2026-06-28, finalised by the team 2026-07-02 — both before the full run). The same model has been evaluated on its own in this literature before [9], which keeps the substitution inside our evidence base. Research questions, metrics, thresholds, and statistical tests were not touched by either amendment.

Baselines. EvoSuite [11] and Randoop [15] generate the Java comparison suites, with 60 seconds of search per class each. That budget was not equal at first — the pilot ran EvoSuite at 60 seconds but Randoop at 10, a bias we only caught when comparing pilot outputs — and was harmonised before the main experiment (Section VI). Pynguin [16] covers Python, running DYNAMOSA with 90 seconds per module. Hypothesis appeared as a candidate baseline in the earliest proposal but was never used for the reported results.

C. Measurement

We chose not to measure functions in isolation. Instead, every generated test runs inside the real project, built and executed at the function's pinned commit (the exact commit hashes, URLs, and download dates are recorded in the dataset's provenance file) — because a large share of the functions in our sample only behave correctly and only resolve the names they reference inside their actual module (Section VI walks through the pilot result that pushed us toward this decision).

Java. Maven 3.9.16 compiles and runs the tests; JaCoCo 0.8.12 collects branch coverage, and PIT 1.17.2 [25] handles mutation testing. Since both tools report at class granularity and our unit of analysis is the individual function, we attribute their output per function by intersecting each report's line numbers with the function's own `[start_line, end_line]` range in the pinned source; the mutation score of a function is the proportion of PIT-generated mutants within that range that its suite kills. Scaling this across 60 functions surfaced two infrastructure bugs of our own: an uncaught PIT timeout that crashed an entire measurement run, and stale `jacoco.xml`/`mutations.xml` reports being picked up for the next function measured in the same module. The second bug would have silently credited one function with another's coverage, so the pipeline now clears report files between runs. GPT-generated Java suites fail to compile far more often than the tool-generated ones do, so each GPT-authored test is compiled and measured individually, with one file per function per run, rather than batching. Batching could result in a working test failing due to a compile error from another test in the same file.

Python. Functions are measured within an editable installation of the pinned `flask` (3.2.0.dev, pinned 2026-05-31) and `requests` (2.34.2, pinned 2026-06-15) checkouts; `coverage.py` 7.15.1 collects branch coverage, cut to the function's line range as on the Java side. Mutation testing uses two instruments, one per arm. The `gpt-4o-mini` suites are scored with `mutmut` 2.4.4, scoped to the target function by marking every line outside its range with `# pragma: no mutate`; only test cases that pass on the unmutated source are executed, and the score is the fraction of non-surviving mutants (killed or timed out) over all mutants generated in the range. The Pynguin suites are scored with a custom engine built on Python's standard `ast` module (no external dependency): it swaps arithmetic ($+ \leftrightarrow -$, $\times \leftrightarrow \div$), comparison ($> \leftrightarrow \geq$, $< \leftrightarrow \leq$, $= \leftrightarrow \neq$), and boolean (`and` $\leftrightarrow$ `or`) operators, negates boolean constants, and increments numeric constants by one — each mutant changes exactly one site inside the function's line range, capped at 20 mutants per function. For every mutant the whole suite is re-run with a 90-second timeout; a mutant counts as killed if the suite fails or times out, and the source file is restored after each run. Functions with no mutable site in range are reported as missing rather than scored 0. Neither pipeline attempts equivalent-mutant detection, so both denominators include all generated mutants and the resulting scores are conservative. Both instruments enforce the same green-on-original gate and the same function-range scoping, but their operator sets differ; the paired RQ2-B comparison between `gpt-4o-mini` and Pynguin therefore crosses instruments, and we flag this explicitly as a construct-validity threat in Section VI (the Java RQ2-B comparisons are unaffected — PIT scores both arms). Restricting mutation to the function's own line range is itself a deliberate constraint: it excludes helper functions and class-level state, which keeps the per-function attribution clean and the two arms comparable, at the cost of not observing faults that only surface through interactions outside the function. Given the pinned checkouts and the fixed model snapshot at `temperature=0`, no step of the measurement pipeline is randomised.

Any suite that fails to compile, import, or execute is marked INVALID: both metrics score 0%, per proposal §5.1. Before mutation analysis, each suite must also pass on the unmutated source. Scoring mutant kills on top of an already-failing suite would quietly inflate the score — Section VI shows the pilot case where exactly that happened — so such suites are rejected outright. An INVALID suite is not the same thing as one that compiles, runs, and still touches none of the target's lines (`compiled=1`, coverage $= 0$): both end up at 0%, for different reasons, and Section IV keeps them separate.

D. Statistical analysis

All tests run at $\alpha = 0.05$, and all of them are non-parametric: coverage and mutation scores are percentages pinned to the $[0, 100]$ interval, so normality was never a defensible assumption for this data. **RQ1** needs one test — a one-sample Wilcoxon signed-rank of branch coverage against the 80% bar. **RQ2** needs two. Whether mutation score clears 60% is again a one-sample Wilcoxon; whether `gpt-4o-mini` beats each baseline is a paired Wilcoxon, matched by `function_id`, with matched-pairs rank-biserial r as the effect size. For **RQ3** we correlate CC with each quality metric using Spearman rank correlation. Every statistic is reported at two levels: *all* ($N = 60$ per language, INVALID

and non-touching functions scored 0 under the pre-registered rule) and *effective* (only the suites that compiled and touched their target). Dropping the all-level would quietly overstate the model, since the effective subset is conditioned on success — so neither level appears without the other (Section V returns to this).

IV. Results

We evaluate one-shot `gpt-4o-mini-2024-07-18` unit tests on 120 functions (60 Java from 8 Defects4J subject programs; 60 Python from `flask`/`requests`) at cyclomatic complexity 5–10, with branch coverage measured by JaCoCo (Java) / `coverage.py` (Python) and mutation score by PIT (Java) / `mutmut` for the LLM suites and a custom AST-level engine for the Pynguin baseline (Python; Section III). All tests run at $\alpha = 0.05$, with matched-pairs rank-biserial r (Wilcoxon) and Spearman ρ (RQ3) as effect sizes. Per the pre-registered scoring rule, any suite that fails to compile or fails to execute scores 0% and is marked INVALID. Every metric below is reported at two levels side by side — *all* ($N = 60$, invalid $= 0$) and *effective* (only the valid suites that actually exercise the target) — since the effective figures are conditional and should never be read without the all-level figures next to them.

A. Test validity (a primary finding)

On the Python side, only **32/60 (53.3%)** of the generated suites executed even one test; the remaining **28/60 (46.7%)** were INVALID. Of the 32 valid suites, just **23** actually exercised the target function (branch coverage > 0); the other 9 ran without ever touching the target code (mock or self-implementation). Breaking the 28 invalid suites down further:

- **23 — `ImportError`** (LLM-attributable): the test imports a symbol that does not exist in the target module (e.g. importing an instance method as a module-level function). A small subset of these are dataset/platform artefacts rather than LLM faults (e.g. `proxy_bypass_registry` is Windows-only; `merge_environment_settings` is a method, not module-level).
- **5 — `TypeError`/`AttributeError`** (environment-sensitive): these may vary with the exact `flask`/`requests` dependency versions and should be confirmed in the canonical pinned environment.

That 46.7% invalidity rate is the first main finding, and the breakdown matters: 23 of the 28 failures are LLM-attributable import errors, not environment noise. The recurring shape — importing an instance method as if it were a module-level function — is a misreading of the API's namespace, not a random slip.

Java tells a different story on the surface: **51/60 (85.0%)** of the generated suites compiled and ran, with only **9/60 (15.0%)** INVALID due to outright compile failure. That lower Java invalidity rate is misleading, though. Of the 51 compiling suites, only **5** actually exercised the target method (branch coverage > 0); the other **46** compiled, ran, and touched *none* of the target's lines. A closer look at a sample of these (`JA-002`, `JA-020`) reveals the dominant pattern: *wrong-target invocation*. The LLM writes a call that is syntactically valid and resolves to a real, accessible method — just not the one it was asked to test. For instance, `CommandLine.getOptionValues(...)` gets called when the sampled function is actually the unrelated, nested `CommandLine.Builder.getOptionValues(...)`; elsewhere, a same-named method inherited from higher up the class hierarchy gets invoked instead. This is a fundamentally different failure mode from Python's `ImportError`s. Here, the Java compiler's own overload/name resolution silently accepts a plausible substitute, so the suite is counted as "compiling." At the same time, it measures nothing about the function it was meant to test. Once both effects are combined, only **5/60 (8.3%)** of Java suites end up effective, compared to 23/60 (38.3%) for Python. Among the 9 truly INVALID suites (non-compiling), 6 target `AccurateMath` — a numeric utility class packed with heavily overloaded, edge-case-dense methods — and 3 target `Gson`'s generically-typed adapter API (`getAdapter`, `getDelegateAdapter`, `fromJson`); just 1 of the 9 (itself an `AccurateMath` function) involves a `private` target method. The failures, in other words, cluster around *API surface* — heavy overloading, generics — far more than around method visibility. That distinction matters for anyone designing a fix: making the target `public` could have mattered in at most 1 of the 9 cases, while a class skeleton — which method exists, what its signature looks like, how it differs from same-named neighbours — addresses the pattern we actually observe. Section V-F tests exactly that.

B. RQ1 — Branch coverage vs. 80%

Across all 60 Python functions the median branch coverage is **0%**: for at least half the sample, the generated suite covered no branch of its target at all. Only 23 of the 60 suites exercised any part of the intended code. The effective subset looks better — median **75.0%** ($n = 23$; 11 of the 23 reach 80%, and 6 hit full coverage) — but still sits under the 80% bar, and the one-sample Wilcoxon finds no support for the threshold ($p = 0.67$, $r = -0.10$). Java repeats the pattern one level lower: all-function median **0%**, effective median **50.0%** on a much smaller subset ($n = 5$, the 8.3% effective rate; $p = 0.91$, $r = -0.60$). Pooling both languages changes nothing: $n = 120$ median **0%**; pooled effective median **75.0%** ($n = 28$, $p = 0.84$, $r = -0.22$). Whichever language or aggregation we pick, GPT-4o-mini stays short of 80% branch coverage (Fig. 2). The effective-subset numbers are conditioned on the model having already succeeded at the hard part — compiling and hitting the right target; the all-level median of 0% is the honest number.

C. RQ2 — Mutation score

(A) vs. 60%. The median mutation score across all 60 functions is **0%**, with a measurable score produced for only 17/60. Restricting to that measured subset — green, passing tests that actually touch the code — the median rises to **36.84%** ($n = 17$; just 4/17 reach $\geq 60\%$), well below the

threshold (one-sample Wilcoxon, $p = 0.99$, $r = -0.66$). Java's all-function median is also **0%** ($n = 60$); even when restricting to the 54 functions where PIT produced a measurable score, the median remains at **0%** (one-sample Wilcoxon, $p = 1.00$, $r = -1.00$) — consistent with the 8.3% effective rate: nearly all Java-generated suites fail to kill any mutants. Pooled across both languages (measured subset, $n = 71$), the median is likewise **0%** ($p = 1.00$, $r = -0.97$). H_0 is not rejected in either language: the tests generated are weak at killing mutants, with Java even weaker than Python — a 0% median even among the 54 functions where PIT generated mutants means those suites execute code without ever placing a bet on what the output should be.

(B) vs. baseline. Paired Wilcoxon tests — gpt-4o-mini against each baseline, matched by `function_id` on mutation score — tell a consistent story. gpt-4o-mini *loses substantially* to both automated Java baselines: against EvoSuite ($n = 54$, $p < 0.001$, $r = -1.00$) and against Randoop ($n = 54$, $p = 0.003$, $r = -1.00$), an effect size at its theoretical maximum, meaning EvoSuite and Randoop win *every* pair where the two methods differ at all (Fig. 1). The direction flips against Pynguin (the Python baseline): gpt-4o-mini *considerably outperforms* it ($n = 13$ paired functions, difference in medians +36.84 pp — GPT median 36.84 vs. Pynguin 0.0 — $p = 0.010$, $r = +0.85$). However, this apparent advantage does not indicate that gpt-4o-mini produces effective tests, but rather illustrates a practical shortcoming: Pynguin's own median branch coverage on this dataset is **0.0%** (Section VI). Neither tool, in other words, reliably produces mutation-killing tests for real code in this band; gpt-4o-mini's nominal win exists only because the baseline collapsed. Under Python 3.10/DYNAMOSA with a 90-second budget per module, Pynguin itself repeatedly times out, hits a `dill` pickling error on modules importing `_json`, or exits via an internal tracer abort, exporting zero test cases for most modules. H_0 is therefore rejected twice, in opposite directions — and the two rejections mean opposite things. The Java result reflects a genuine quality gap: EvoSuite and Randoop systematically produce tests that kill mutants, and gpt-4o-mini does not. The Python result reflects a tooling failure. Reading both as "the model beats some baselines but not others" would be the wrong takeaway.

D. RQ3 — Complexity vs. quality

For Python's effective subset, Spearman $\rho = +0.18$ ($p = 0.41$, $n = 23$) shows *no* significant negative correlation between cyclomatic complexity and branch coverage inside the CC 5–10 band. Java's effective subset ($n = 5$) yields $\rho = +0.13$ ($p = 0.83$); again, no significant correlation, though the sample is tiny. Pooled ($n = 28$), $\rho = +0.16$ ($p = 0.40$). H_0 is not rejected in either language: inside the narrow CC 5–10 band we sampled, complexity does not predict coverage quality among the functions that produce any signal at all — what dominates instead is the binary compiles/touches-target outcome discussed above, not a graded decline as CC rises. The pre-registered decision rule for RQ3 was $\rho < -0.5$ (at $p < 0.05$); the observed correlations fail it by sign alone — if anything, positive values are the wrong sign for the complexity-hurts-quality hypothesis, though none is distinguishable from zero.

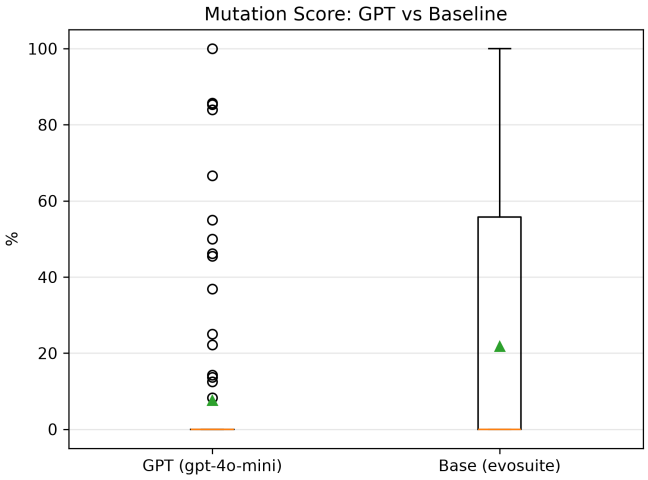

Fig. 1. Mutation score, gpt-4o-mini vs. EvoSuite, matched by function ($n = 54$ paired Java functions). EvoSuite wins in every pair where the two differ ($r = -1.00$, $p < 0.001$).

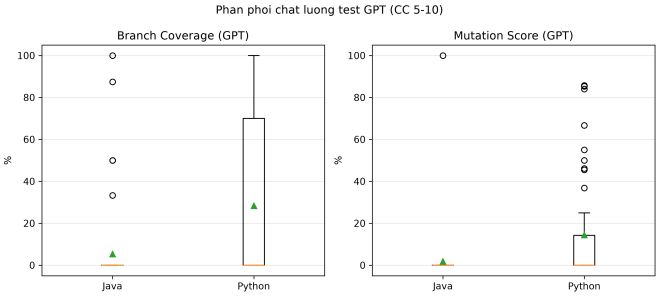

Fig. 2. Branch coverage and mutation score of gpt-4o-mini on the effective subset (dashed: 80% / 60% thresholds). The all-function medians are 0% (see text).

E. Summary of findings

Neither the 80% coverage nor the 60% mutation threshold is met in either language ($N = 120$). Even in the most favourable group, the effective subset, coverage is 75.0%, still below 80%. When counting all 120 functions, the median is 0%. There is no clear link between complexity and quality within CC 5–10 for either language. What matters most here is test validity, and it breaks differently in each language: Python mostly dies at compile time (46.7% INVALID, dominated by `ImportError`), while Java mostly compiles and then measures nothing (76.7% compile yet touch 0% of the target), through silent wrong-target/overload resolution. Effective suites make up only 8.3% of the Java sample and 38.3% of the Python one. On paired comparisons, gpt-4o-mini loses decisively to both automated Java baselines (EvoSuite, Randoop; $r = -1.00$ each) but beats the Python baseline

TABLE I
PER-RQ SUMMARY (GPT-4O-MINI). "ALL" = 60/LANG. (INVALID/NO-TOUCH = 0); "EFF." = EFFECTIVE SUBSET (COMPILED & TOUCHES TARGET); "MEAS." = SUBSET WITH A MEASURABLE MUTATION SCORE.

RQ	Metric	Level	Median	p	Effect
RQ1 Py	Coverage≥80	all (60)	0%	–	–
		eff (23)	75.0%	0.67	r=−0.10
RQ1 Java	Coverage≥80	all (60)	0%	–	–
		eff (5)	50.0%	0.91	r=−0.60
RQ1 All	Coverage≥80	all (120)	0%	–	–
		eff (28)	75.0%	0.84	r=−0.22
RQ2A Py	Mutation≥60	all (60)	0%	–	–
		meas. (17)	36.84%	0.99	r=−0.66
RQ2A Java	Mutation≥60	all (60)	0%	–	–
		meas. (54)	0%	1.00	r=−1.00
RQ2B Java	GPT vs EvoSuite	paired (54)	–	<0.001	r=−1.00
RQ2B Java	GPT vs Randoop	paired (54)	–	0.003	r=−1.00
RQ2B Py	GPT vs Pynguin	paired (13)	Δmed +36.84pp	0.010	r=+0.85
RQ3 Py	CC∼Cov	eff (23)	ρ=+0.18	0.41	+0.18
RQ3 Java	CC∼Cov	eff (5)	ρ=+0.13	0.83	+0.13
RQ3 All	CC∼Cov	eff (28)	ρ=+0.16	0.40	+0.16

Pynguin, whose own coverage on this dataset sits at 0.0% because of tool–environment issues rather than any real task difficulty (Table I).

F. Threats to validity

Conditional subset. The effective-level medians (75%/50%/37%/0%) are conditioned on suites that both compiled *and* touched the target, so they *overstate* raw ability relative to the all-level (0%) — which is why we report both at every RQ rather than either alone. **Environment.** The Python figures come from a clean editable-install environment; the exact invalid rate is sensitive to the precise `flask`/`requests` dependency versions in use. The Python baseline figures needed a separate Python 3.10 virtual environment just for Pynguin, since Pynguin's bytecode instrumentation silently produced empty test suites under Python 3.14, the platform otherwise used for GPT/measurement scripting; this is documented in Section VI as an environment amendment, not as a change to any pre-registered metric or threshold. **Sample size.** Effective n is small in every cell (5–28), and particularly so for Java RQ1/RQ3 ($n = 5$) — those point estimates are indicative, not precise. **Single sample.** With one generation per function (one-shot, temp = 0), we have no per-function variance estimate. **Cross-language tooling.** The Python harnesses (`coverage.py`, `mutmut`, the AST mutation engine) and the Java ones (JaCoCo, PIT) differ enough in implementation that cross-language comparisons in this section stay descriptive — we never test one language's scores against the other's. **Baseline-tool confound.** The RQ2B win against Pynguin reflects Pynguin's own tooling trouble on this environment (Section VI), not evidence that gpt-4o-mini produces strong tests in absolute terms.

V. DISCUSSION

A. Coverage is not test quality

Branch coverage and mutation score pull apart in our own data, on a controlled sample — the very pattern that motivated this study's paired metrics. `JA-008` is the clearest case: a baseline suite reaches 60% branch coverage while killing 0% of mutants. The suite runs the function, visits its branches, and exits without ever asserting anything that would distinguish a correct return value from a mutated one. It is not a broken test — it just has no opinion about the code. The aggregate RQ1/RQ2 numbers split the same way. Effective-subset coverage (75.0% Python, 50.0% Java) reads as far less alarming than the mutation numbers over the measurable subsets (36.84% Python, 0% Java). Someone shown only branch coverage would call GPT-4o-mini's tests moderately competent; someone shown only mutation score would call them close to worthless. Both descriptions are accurate; the metric you choose determines the conclusion you reach, which is exactly why we report both, at both the *all* and *effective* levels.

B. Where does the "breaking point" actually sit?

GAP-T asked for the cyclomatic-complexity threshold at which LLM test generation starts to degrade. RQ3 finds no significant CC–quality correlation within CC 5–10 in either language ($\rho \in [+0.13, +0.18]$, all $p > 0.4$), and next to RQ1/RQ2 that result argues against the "breaking point" framing itself: quality does not decline *gradually* across CC 5–10 at all. What dominates is a *binary* outcome, decided almost independently of CC within this band — whether the generated suite compiles, and whether it invokes the sampled method rather than a same-named neighbour or an inherited overload. A CC 5 function and a CC 10 function in our sample are roughly equally likely to fail that gate. Complexity may still matter to LLM test generation in general; inside the narrow, pre-selected medium-complexity band we sampled (chosen to stay clear of the extremes), something else drives most of the variance. Pinning down a true breaking point would require sampling low-CC (< 5) and high-CC (> 10) functions for contrast, which we leave as future work (Section VII).

C. Cost is not the primary constraint; correctness is

Generating all 120 one-shot suites cost well under a dollar in API charges (the pre-registered budget estimated $1;

amendment v1.1, §8.2) and took only a few minutes. EvoSuite and Randoop each ran a 60-second search budget per class, so covering the same 60 Java functions took tens of minutes before any measurement began. On price and speed, the LLM (`gpt-4o-mini-2024-07-18`) wins outright. On exercising and killing mutants on the intended target, it loses to both automated Java baselines with an effect size of $r = -1.00$ — every non-tied pair favours the baseline. Cheap and fast, then, but not reliable, and the gap is wide. Spending part of the saved time on a few compile-and-retry attempts (a feedback loop) could narrow it without approaching EvoSuite or Randoop's total runtime.

D. The INVALID rate as a finding, not a bug

We did not filter out functions that are structurally hard for a one-shot external test to reach — `private` target methods being the obvious example, at 17/60 of the Java sample. Removing them after seeing Java's low effective rate would have meant selecting the sample on the very outcome under measurement, against the pre-registered protocol (amendments v1.1/v1.2, §VI). The data then showed that such a filter would have cut the wrong variable anyway: visibility is *not* the dominant cause. A `private` target is involved in only 1 of the 9 truly non-compiling Java functions and only 14 of the 46 that compiled but touched nothing (Section IV). The dominant failure mode — wrong-target invocation via silent overload/name resolution — would have gone unexplained; the outright compile failures cluster separately, around the API-heavy `AccurateMath` and `Gson` classes (Section IV).

Nor is the pattern unique to our sample. Haratian et al. [6] report a 63% compilation-error rate for GPT-4o on 353 real Java pull requests — roughly two-thirds of our Java INVALID-or-no-touch rate (91.7%), and the same qualitative story: one-shot, zero-context generation on real, uncurated code compiles far less reliably than benchmark-style evaluations suggest. Sapozhnikov et al. [7] report over 50% malformed or non-compiling ChatGPT-generated Java tests, and Kumar et al. [2] trace weak single-pass results directly to the *absence of an iterative-repair loop*. The fixes the literature reaches for — feeding compiler or test-failure output back to the model (Zeng et al. [17]; Xue et al. [19]), retrieving real API signatures instead of raw code (which targets exactly the wrong-target-invocation pattern we observe), or sampling several candidates and keeping whichever compiles — all step outside the strict one-shot setting evaluated here. Our numbers are therefore a lower bound on what GPT-4o-mini can do with this codebase, and a direct demonstration of why the field has moved toward feedback-augmented and retrieval-augmented pipelines rather than static one-shot prompting.

E. Why gpt-4o-mini "beats" Pynguin

The RQ2-B result against Pynguin ($r = +0.85$, favouring gpt-4o-mini) says more about the baseline than about the treatment. Pynguin's own median branch coverage on this dataset is 0.0% (Section VI), driven by search timeouts and a `dill` serialisation failure on modules that import `_json` — neither of which touches gpt-4o-mini's ability to write tests. We report the comparison anyway: it is the pre-registered baseline for Python (amendment v1.2), and quietly dropping an inconvenient result would be its own validity threat. It is not evidence that gpt-4o-mini writes good Python tests in absolute terms — RQ1/RQ2-A already rule that out.

F. RQ4 (exploratory, post-hoc): does real API context help?

Wrong-target invocation, the dominant Java failure above, motivated a small *exploratory, post-hoc* follow-up: the same 120 functions, the same model/temperature/exemplar, but with the target class's (Java) or module's (Python) real public-API skeleton — method signatures, nested-class structure, the distinction between a module-level function and a class method — extracted from the pinned source with `javalang`/`ast` and inserted into the one-shot prompt. The idea came only after the $N = 120$ results were in, so it sits outside the pre-registered RQ1–RQ3 and is reported separately; folding it into the confirmatory analysis would be HARKing. The new suites went into a separate directory and results file, leaving the registered $N = 120$ data untouched.

Java. The compile rate does not move (51/60 in both arms), but the *effective* rate — suites that actually exercise the target — more than doubles: 11/60 (18.3%) with context versus 5/60 (8.3%) without. Paired by function, 7/60 improve, 1/60 regresses, and 52/60 stay unchanged; a one-sided paired Wilcoxon test falls just short of significance at $\alpha = 0.05$, though 7 favourable pairs against 1 unfavourable is suggestive on its own, and the effect size among the non-tied pairs is moderate-to-large. Mutation score does not improve at all ($n = 48$ pairs; both medians 0%). Context, then, fixes the structural gap — which method to call — and leaves the behavioural gap untouched: the model still does not assert what the method should return, only that it can be called. The two problems need different interventions. Wrong-target invocation is an API-navigation failure; weak assertions are a test-design failure. A class skeleton addresses the first; the second likely needs examples of what a meaningful assertion looks like for the specific function under test.

Python. Judged by the pre-registered whole-suite-must-pass criterion, context looks *worse* (7/60 compiled versus 32/60) — a measurement artefact, not evidence of harm. Counted at the level of individual test functions rather than whole files, the two arms are near-identical: 267 test cases at 33.3% passing with context, versus 233 test cases at 32.2% passing without. What context actually does is push gpt-4o-mini to write *more* test cases per file — plausibly because it now sees more of the API surface — and since our Python INVALID criterion demands that every test in a file pass, more tests per file mechanically raise the odds that at least one of them trips over an unrelated behavioural bug. We traced one such case, `flask.logging.has_level_handler`: an identical misunderstanding of logger-hierarchy propagation appears in *both* arms, unrelated to context at all. That leaves a second-order validity threat for future work: an all-or-nothing pass criterion penalises test-suite thoroughness the moment

any single assertion is wrong, and it can make two arms with near-identical per-test correctness look dramatically different.

Implication. Static API context costs little — no additional model calls beyond the first one-shot budget. It has a real, if not yet significant, effect on Java's wrong-target problem, and a neutral effect on Python once measured fairly. Neither result changes the RQ1–RQ3 conclusions (Section IV): with or without context, gpt-4o-mini one-shot generation stays well short of the 80%/60% thresholds.

G. RQ5 (exploratory, post-hoc): one feedback/repair iteration on top of RQ4

We committed to a single repair round before running it, and ran it exactly once — iterating a technique until some threshold clears is precisely the drift our protocol rules out. Every RQ4 suite that was not yet effective went back to gpt-4o-mini with a real failure signal and a request for one corrective rewrite. For Java the signal was the 0%-coverage outcome plus the target's API skeleton, since Maven/JVM error logs proved too noisy to serve as a clean compiler-diagnostic signal; for Python it was the actual fresh `pytest` failure output. Suites already effective at RQ4 (11/60 Java, matching Python's per-test baseline) were carried forward unchanged rather than regenerated.

Java: further, still-inconclusive improvement. The effective rate climbs again, from 11/60 (18.3%, RQ4) to **15/60 (25.0%)**. Among the functions that changed, 4 improved and *none regressed* — repair matched or beat context on every paired function. The one-sided paired Wilcoxon test gives $p = 0.063$, still short of $\alpha = 0.05$ at this sample size, with a rank-biserial effect size of $r = 1.00$ (every non-tied pair favours repair) and a monotonic climb across all three stages ($8.3\% \rightarrow 18.3\% \rightarrow 25.0\%$). Mutation score stays flat at a median of 0% ($n = 45$ paired functions), exactly as it was at RQ4. Repair gets GPT-4o-mini to hit the right lines more often; just as with context alone, the assertions still are not strong enough to kill mutants.

Python: a genuine regression, not a measurement artefact this time. Per-test-case pass rate *falls* to 15.5% (106/682 tests), below both RQ4 (33.3%) and the first no-context arm (32.2%). File count and suite length do not explain this one, unlike the RQ4-vs-original gap. Test volume grew again — 682 tests across 60 files, from 267 at RQ4 — so gpt-4o-mini appears to respond to a visible failure by *expanding* the suite rather than narrowly fixing the assertion that failed, and the added cases fail at a higher rate than the first ones survived at. We do not have a definitive mechanism, and the run-once commitment rules out chasing one post hoc. Whether the more-tests-not-better-tests response is a property of gpt-4o-mini specifically, of single-shot repair in general, or of the particular failure signals we fed it, remains open.

Overall verdict on RQ4+RQ5. Two rounds of low-cost, non-fine-tuning intervention leave Java with a real, if not significance-clearing, upward trend in coverage effectiveness and no downside anywhere, and Python with no benefit from context plus an outright regression from single-shot repair. Neither technique closes the gap to the pre-registered 80%/60% thresholds, alone or combined (Section IV): across the full $N = 120$ sample, the median stays at 0% for both metrics at every stage we tried. The interventions the literature recommends as next steps after one-shot prompting (Section II) are, on this evidence, no guaranteed fix: they help unevenly across languages, and in at least one case here (Python repair) they make things worse. "Add more scaffolding" does not straightforwardly improve LLM test generation. We agreed in advance not to keep searching for a technique that crosses the threshold, so we stop here rather than attempt a third intervention.

VI. Threats to Validity

A. Construct Validity

Three times during this study a measurement produced a wrong result while the tooling signalled success, and each time only a cross-check against the artefacts on disk caught it. The first: mutation scores of 100% for test suites that had in fact *failed on the unmutated code* — every mutant was trivially "killed" because the suite was already broken. A green-on-original check now runs before any mutation analysis; a test that fails on the baseline source is scored INVALID outright. The second: EvoSuite returns exit code 0 even when it generates no tests whatsoever, and a run of 12 functions was initially logged as a complete success with zero test files on disk. The pipeline now checks for the generated file itself rather than trusting the exit code. The third: JaCoCo silently reported 0% coverage for EvoSuite tests, because EvoSuite's separate instrumenting class loader changes class identity underneath it; disabling `separateClassLoader` during measurement fixed the attribution.

A fourth construct threat is disclosed rather than fixed: the two Python arms are scored by different mutation instruments — `mutmut` for the `gpt-4o-mini` suites, our custom AST engine for the Pynguin suites (Section III). Both enforce the same green-on-original gate and the same function-range scoping, but their operator sets differ, so the Python RQ2-B pairing compares scores produced by two different instruments. Given that Pynguin's median coverage on this dataset was 0.0%, the direction of that comparison does not hinge on the instrument choice. Our own data also show why coverage alone is an unreliable proxy for test quality: one baseline suite reached 60% branch coverage on a function while killing 0% of its mutants, which is exactly why we report the coverage–mutation pair rather than either alone.

B. Internal Validity

The pilot run existed to surface integration defects before the main study, and it did. The first version of the Java dataset contained function files whose contents did not match their metadata — `JA-002`'s file held an unrelated `concat` method instead of `getOptionValues` — so the LLM was shown source code unrelated to the specified method. Every pilot LLM test was discarded and regenerated once the dataset was corrected. On the Python side, 10 out of 12 pilot measurements came back INVALID because functions

had been extracted without their module context, an error unrelated to the LLM; measurement moved into real installed modules at pinned commits as a result. Baseline generation had its own defects: early experiments gave EvoSuite a 60-second search budget against Randoop's 10 seconds, and Randoop's output files were overwritten across runs. Both were fixed before the main experiments — equal budgets, per-function output names. Several tool–environment mismatches also had to be resolved: EvoSuite 1.2.0 ran under JDK 11 for most classes but needed a Java 8 runtime for the `csv`/`gson`/`joda` classes, so the generated suites mix JDK 11 (39 functions) and JDK 8 (21 functions) provenance; multi-release dependency JARs carried Java 21 bytecode that older ASM versions reject (`Unsupported class file major version 65`); and `commons-math` disables JaCoCo in its own pom via `jacoco.skip=true`. Four out of six measurement-pipeline failures terminated with exit code 0. Silent failures of that kind survive anything short of checking that the result artefacts actually exist.

C. External Validity

Every result here characterises the lightweight `gpt-4o-mini-2024-07-18` and only that model; extrapolating to larger models such as GPT-4o is unwarranted (amendment v1.1). The dataset spans two ecosystems — Apache-style Java libraries and Python web libraries (flask, requests) — and is unevenly distributed within Python: 48 of 60 functions come from flask. Functions that depend on heavyweight environmental context, such as GUI or network access, are under-represented. Baseline coverage also varies sharply by repository, running near-perfect on numeric utility code but substantially lower on parser- and formatter-style classes, so a threshold like RQ1's 80% target is repository-relative rather than a universal bar.

D. Conclusion Validity

With $N = 120$ functions (60 per language), we rely throughout on non-parametric tests — one-sample and paired Wilcoxon, Spearman correlation, $\alpha = 0.05$ — and report effect sizes alongside every p-value. The thresholds (80% coverage, 60% mutation) were fixed in the approved proposal before any full-run data existed, and every protocol change made along the way (model, dataset, prompting mode, measurement context) was recorded as an amendment before the corresponding data were produced — that ordering is what mitigates HARKing here. Mutation scores for 3/60 baseline cells — `JA-017` (`jfreechart`), `JA-048` and `JA-058` (both `jsoup`) — were unavailable: PIT generated no mutants within these functions' target line ranges under either baseline tool, so there was nothing to score. Branch coverage for these 3 functions is unaffected, since it is measured independently and before the PIT step, and remains in `metrics_full.csv`; only the mutation-score cell is reported as missing rather than imputed as zero, so the baseline's RQ2 numbers are not silently deflated. The same three functions also appear among the LLM's own INVALID/non-effective Java results (Section IV), for an entirely unrelated reason: wrong-target invocation. PIT's missing mutants and the LLM's wrong-target problem were diagnosed separately; the overlap is coincidence.

A separate environment issue hit only the Pynguin baseline (Python). Under Python 3.14, the interpreter used everywhere else in the measurement pipeline, Pynguin's bytecode instrumentation produced empty test suites (0.0% coverage) for every module. Reproducing the failure with verbose logging traced it to Pynguin's internal master–worker subprocess — a genuine tool–interpreter incompatibility rather than a configuration mistake. The fix was a dedicated Python 3.10 virtual environment for Pynguin invocations only, the interpreter version most Pynguin releases are actually validated against. It changes none of the pre-registered metric, threshold, dataset, or model, and it was applied uniformly before a single Pynguin measurement was taken, so it carries no HARKing risk. Even with the fix in place, Pynguin's own median branch coverage on this dataset stayed at 0.0% (Section IV), which we read as a genuine, if weak, finding about Pynguin's real-world difficulty on these modules rather than a leftover environment defect.

VII. Conclusion

This study measured GPT-4o-mini as a one-shot unit-test generator on 120 real-world Java and Python functions of medium cyclomatic complexity (CC 5–10), scoring every suite inside its host project and comparing against EvoSuite and Randoop (Java, equal 60-second budgets) and Pynguin (Python, 90-second budget).

RQ1 (coverage threshold). GPT-4o-mini does *not* achieve a median branch coverage $\geq 80\%$ at any level of analysis. Across all 120 sampled functions the median is 0% (H_0 not rejected, one-sample Wilcoxon). Even restricted to the 28 functions whose suites both compiled *and* exercised the target, the median stops at 75.0% ($p = 0.84$, $r = -0.22$). The bar itself is demanding: on the same 60 Java functions EvoSuite manages a median branch coverage of 55.0% and Randoop 21.1%, so no tool clears 80% on this CC 5–10 sample.

RQ2 (quality vs. baselines). Mutation scores are weaker still: over the suites with a measurable mutation score, the median is 36.84% for Python ($n = 17$) and 0% for Java ($n = 54$), both well under the 60% threshold. Paired Wilcoxon tests hand both Java baselines a decisive win — EvoSuite at $p < 0.001$, Randoop at $p = 0.003$, each with $r = -1.00$: not one function where the two suites differed went to GPT-4o-mini. Against Pynguin the comparison flips ($p = 0.010$, $r = +0.85$ in GPT-4o-mini's favour), but Pynguin recorded just 0.0% median coverage on this dataset — a tooling failure examined in Section VI — so the win says nothing about the robustness of GPT-4o-mini's own tests.

RQ3 (complexity correlation). Cyclomatic complexity shows no significant correlation with coverage inside the CC 5–10 band in either language (Python $\rho = +0.18$, $p = 0.41$; Java $\rho = +0.13$, $p = 0.83$; pooled $\rho = +0.16$, $p = 0.40$). Within this band, test effectiveness hinges on a binary outcome — whether the generated suite compiles and invokes the correct target (Section IV) — rather than on

any continuous relationship with complexity. Locating the true "breaking point" would take a sample that extends below CC 5 and above CC 10.

Two lessons stand out. First, evaluating LLM-generated tests on functions extracted out of context understates the difficulty of real-world code: functions from mature projects lean on their surrounding module, and an evaluation that ignores that context flatters the model. Second, the measurement pipeline deserves the same scrutiny as the model. Most of the failures we hit were silent — exit code 0, no artefact — so every step had to be checked against a report file that actually exists on disk (Section VI). Coverage alone is likewise too coarse a signal; without the paired mutation analysis, apparently adequate suites would have been overrated.

Future work. RQ4 and RQ5, two exploratory post-hoc follow-ups (Sections V-F–V-G), asked whether low-cost prompting interventions could close the gap to the pre-registered thresholds. They could not. Java's effective-suite rate did climb across the three stages — 8.3% $\rightarrow$ 18.3% $\rightarrow$ 25.0% with no intervention, real API context, and one feedback/repair iteration respectively — one-sided $p = 0.074$ for the context step and $p = 0.063$ for the repair step, both short of significance at $n = 60$, while mutation score stayed at 0% throughout. Python gained nothing from either the added context or the feedback/repair step; the single-shot repair in fact produced a clear decline, most likely because gpt-4o-mini answered failures by padding the suite with more tests rather than by fixing the failed assertions. We stopped the exploratory arm at these two interventions and did not keep hunting for a technique that would cross the preset thresholds (anti-HARKing).

Five concrete directions remain: (1) expanding the Java sample to test whether the positive coverage trend from RQ4/RQ5 reaches statistical significance; (2) evaluating multi-iteration repair loops with explicit stopping criteria based on suite size, to determine whether Python's regression is a property of single-shot repair or persists under further refinement; (3) replicating the analyses with larger LLMs (GPT-4o or Claude-class models) to isolate the role of model capacity; (4) broadening the dataset to additional Python libraries and to functions of higher cyclomatic complexity (CC $>$ 10) to locate potential breaking points more precisely; and (5) integrating the generation–measurement pipeline into continuous integration workflows for ongoing, in situ assessment. All five call for pre-registered protocols.

On this evidence, one-shot LLM test generation is not yet a substitute for EvoSuite or Randoop on real-world code. Closing that gap will take feedback, retrieval, and repair mechanisms on the generation side, and context-aware, artefact-verified benchmarks on the evaluation side.

AI Use Disclosure

This document was originally drafted by human authors. Automated evaluation and reporting sections, including test generation and analysis, were produced and refined using AI-based large language models.

References

[1] W. Wang, C. Yang, Z. Wang, Y. Huang, Z. Chu, D. Song, L. Zhang, A. R. Chen, and L. Ma, "TestEval: Benchmarking large language models for test case generation," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 3547–3562.

[2] A. Kumar *et al.*, "Empirical evaluation of advanced LLMs on automated unit test generation for Java applications," *IEEE Access*, vol. 13, pp. 1245–1260, 2025.

[3] Lira *et al.*, "Evaluating the effectiveness and cost-efficiency of large language models in automated unit test generation," in *Anais do Simpósio Brasileiro de Qualidade de Software (SBQS)*, 2025. [Online]. Available: https://sol.sbc.org.br/index.php/sbqs/article/view/39000

[4] A. Chudic and G. Çalıklı, "Automated test suite enhancement using large language models with few-shot prompting," 2026.

[5] K. Jasti, T. P. Sahu, R. Pentyala, M. Mohit, and V. Yelleti, "FeedbackLLM: Metadata-driven multi-agentic language-agnostic test case generator with evolving prompt and coverage feedback," 2026.

[6] M. Haratian *et al.*, "PR-aware test case generation: An empirical comparison of search-based and LLM-based approaches," 2026.

[7] G. Sapozhnikov *et al.*, "An empirical study on the structural coverage and compilation rate of ChatGPT-generated unit tests," in *Proc. ICSE Companion*, 2024.

[8] Q. Zhang, Y. Shang, C. Fang, S. Gu, J. Zhou, and Z. Chen, "TestBench: Evaluating class-level test case generation capability of large language models," 2024.

[9] D. Konstantinou *et al.*, "Evaluating plain-LLM test generation across diverse Java projects: A multi-model analysis," 2026.

[10] R. Jain and C. Le Goues, "TestForge: Feedback-driven, agentic test suite generation," 2025.

[11] G. Fraser and A. Arcuri, "EvoSuite: Automatic test suite generation for object-oriented software," in *Proc. ESEC/FSE*, 2011.

[12] W. C. Ouédraogo *et al.*, "Prompt engineering in LLMs for automated unit test generation: A large-scale study," *Empirical Software Engineering*, 2025.

[13] S. Gu *et al.*, "Improving LLM-based unit test generation via template-based repair," 2025, aCM Trans. Softw. Eng. Methodol. (TOSEM).

[14] L. Broide, R. Stern, and A. Mordoch, "EvoGPT: Leveraging LLM-driven seed diversity to improve search-based test suite generation," 2026.

[15] C. Pacheco and M. D. Ernst, "Randoop: Feedback-directed random testing for Java," in *Proc. OOPSLA Companion*, 2007.

[16] S. Lukasczyk and G. Fraser, "Pynguin: Automated unit test generation for Python," in *Proc. ICSE Companion*, 2022.

[17] Zeng *et al.*, "A logic-guided and explainable approach to LLM-based unit test generation," *Applied Sciences*, vol. 16, no. 5, p. 2542, 2026.

[18] M. Konstantinou, R. Degiovanni, J. M. Zhang, M. Harman, and M. Papadakis, "YATE: The role of test repair in LLM-based unit test generation," 2025.

[19] P. Xue, Y. Zhang, Z. Yang, X. Ren, X. Li, P. Hu, L. Wu, and K. Li, "DISTINCT: A description-guided branch-consistency analysis framework for non-regressive test case generation," 2025, iEEE Trans. Softw. Eng. (TSE).

[20] R. Liu *et al.*, "Type-aware LLM-based regression test generation for Python programs," 2025.

[21] G. Wang, C. Yang, X. Zhou, Z. Sun, B. Wang, D. Lo, and D. Hao, "TestDecision: Sequential test suite generation via greedy optimization and reinforcement learning," 2026.

[22] A. Moradi Dakhel, A. Nikanjam, V. Majdinasab, F. Khomh, and M. C. Desmarais, "Effective test generation using pre-trained large language models and mutation testing," 2023.

[23] G. Antal *et al.*, "Leveraging GPT-4 for vulnerability-witnessing unit test generation," in *Proc. EASE*, 2025.

[24] Silva *et al.*, "LLMs as test generators: A comparative benchmarking study," in *Anais do Simpósio Brasileiro de Engenharia de Software (SBES)*, 2025. [Online]. Available: https://sol.sbc.org.br/index.php/sbes/article/view/36983

[25] H. Coles, T. Laurent, C. Henard, M. Papadakis, and A. Ventresque, "PIT: A practical mutation testing tool for Java," in *Proc. ISSTA*, 2016.